

Project Brief: "Vault" — Multi-Tenant AI Knowledge Platform

The pitch (read this like a real product brief)

Teams accumulate documents — PDFs, meeting notes, wikis — and nobody can find anything in them six months later. **Vault** is a backend service where a team uploads documents, and any team member can ask natural-language questions and get answers grounded in their team's own documents, with sources cited.

This is not "another RAG demo." The engineering difficulty is everything *around* the RAG core: multiple teams sharing one system without seeing each other's data, documents being ingested reliably at scale, the system staying observable and debuggable in production, and API consumers (a future frontend, or another service) having a stable contract to build against.

Why this project, specifically

It's the direct evolution of your strongest existing project (Local PDF RAG), but adds exactly the dimensions that are currently missing from your whole portfolio:

Skill gap today	How this project forces it
No multi-tenancy / auth-scoped data anywhere	Every query must be scoped to a team — get this wrong and team A reads team B's private docs
No testing discipline	Ingestion pipeline and auth boundaries are exactly the kind of logic that silently breaks without tests
No production observability	You'll need real metrics: ingestion latency, query latency, failed jobs
No system design practice	You'll have to make real decisions: sync vs async ingestion, how to handle a 500-page PDF, what happens when the vector DB is down
Nothing deployed live	This should end with a real, running instance you can link to

Non-negotiable requirements (this is the actual scope)

1. Multi-tenancy

- Users belong to an "organization" (team)
- A user can only ever query/see documents belonging to their own organization — this must be enforced at the query layer, not just hidden in the UI
- JWT-based auth, org ID embedded in the token claims

2. Document ingestion pipeline

- Upload endpoint accepts PDFs
- Ingestion (chunking + embedding + vector storage) runs as a **background job**, not inline in the request — you already know why from your Book Catalog project
- Job status must be queryable: "processing," "ready," "failed" — a user should be able to check on their upload
- Handle failure gracefully: what happens if a PDF is corrupted, or embedding generation times out? Don't let one bad file crash the worker silently

3. RAG query endpoint

- Given a question, retrieve only from the requesting org's documents, generate a grounded answer, and **return the source document(s)** the answer came from — no unsourced answers
- Reuse your local-model approach (Ollama) so this stays runnable without external API costs, but structure the code so swapping in a hosted LLM later is a config change, not a rewrite (this is a real system design skill: designing for swappable dependencies)

4. Observability

- Add basic metrics: ingestion job duration, query latency, number of failed ingestion jobs
- Expose them in a way Prometheus could scrape (you already have this on your skills list — now actually wire it up once)

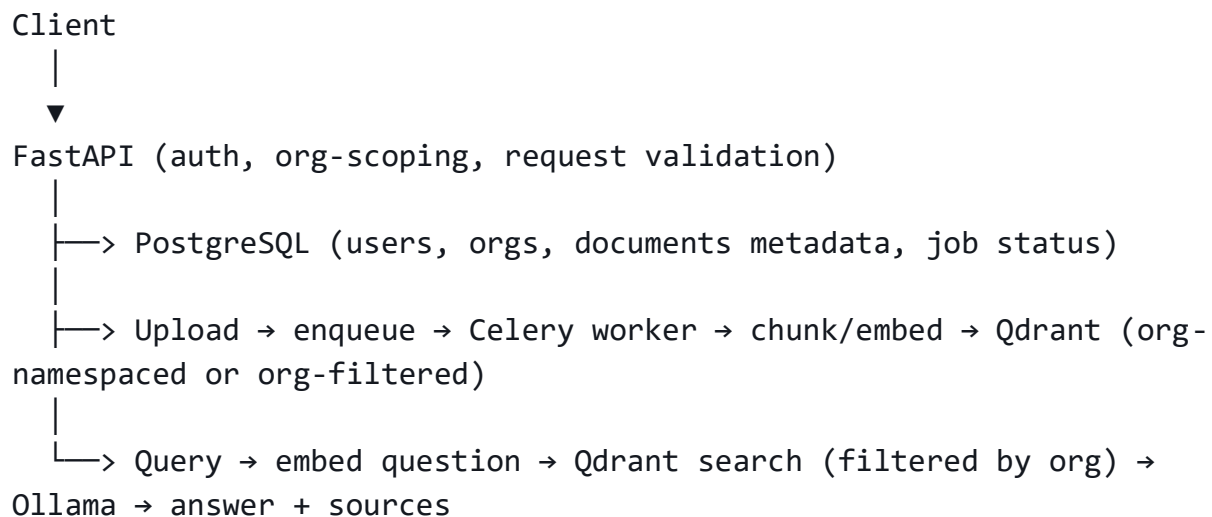
5. Testing

- Auth/tenancy boundary must have tests proving org A genuinely cannot retrieve org B's documents — this is the single most important test in the whole project
- Ingestion pipeline tests with a mocked embedding step (don't call a real model in tests — mock it)

6. Deployment

- Docker Compose for local dev
- Deployed somewhere real (Railway/Render/Fly.io) with a live URL you can put in your CV/portfolio

Suggested architecture (yours to challenge — a real tech lead expects pushback, not blind compliance)



Sprint plan — work through these in order, don't skip ahead

Sprint 1: Foundations

- Org and User models (PostgreSQL), with a user belonging to exactly one org for now (multi-org-per-user is a stretch goal, not v1)

- JWT auth: signup, login, token includes `org_id`
- A `require_org_scope` dependency in FastAPI that every protected route uses — this is your single point of tenancy enforcement, make it impossible to bypass by accident
- **Acceptance criteria:** two orgs exist in your dev DB; user from org A gets a 403 (not empty results — 403) when trying to hit any endpoint scoped to org B's resource ID directly

Sprint 2: Ingestion pipeline

- Upload endpoint → stores file, creates a "document" row with status pending, enqueues a Celery job
- Celery worker: chunk → embed → store in Qdrant with `org_id` as metadata for filtering
- Update document status to ready or failed (with an error message) when the job finishes
- Endpoint to check a document's ingestion status
- **Acceptance criteria:** upload a real PDF, watch status go pending → ready, confirm chunks exist in Qdrant tagged with the right `org_id`. Upload a corrupted file on purpose and confirm it fails gracefully with a readable error, not a crashed worker.

Sprint 3: Query endpoint

- Query endpoint: embed question, search Qdrant **filtered by the requester's `org_id`**, pass top chunks + question to Ollama, return answer + which document(s) it came from
- **Acceptance criteria:** create two orgs, each with different documents. Confirm org A's queries never surface org B's content, even for very similar/generic questions. Write this as an actual test, not just manual spot-checking.

Sprint 4: Observability + tests

- Add timing metrics around ingestion and query paths
- Expose a `/metrics` endpoint (Prometheus format)
- Write the tenancy-boundary test suite properly (this is the one I'd review most carefully)
- Write ingestion pipeline tests with mocked embedding calls

Sprint 5: Deploy + polish

- Docker Compose for full local stack
- Deploy live
- Write the README the way we did for your other repos — overview, architecture diagram, honest limitations, possible next steps
- Record a 2-minute screen-recording demo (uploading a doc, asking a question, seeing sourced answer) — this is a huge upgrade for a portfolio project and takes 10 minutes to make

How we'll work together on this

Bring me things at these points, and I'll respond like a reviewer, not just a cheerleader:

1. **Before you code a sprint** — tell me your planned approach for the trickiest part (usually the tenancy enforcement or the async job handling) and I'll poke holes in it before you build the wrong thing
2. **When you're stuck** — paste the actual error or the actual code, not just "it doesn't work"
3. **When a sprint is done** — share the code or a diff and I'll review it like a PR: what's solid, what I'd push back on, what a stricter reviewer at a bigger company would flag

One more thing worth saying: **don't rush this**. A rushed version of this project is just RAG-project-v2 with extra steps. A properly built one — where you can defend every design decision, including ones you'd do differently now — is the single strongest thing on your entire portfolio when it's done. Take the time Sprint 1 needs to actually be solid before moving to Sprint 2; tenancy bugs discovered late are painful to retrofit.

Ready when you are — tell me which sprint you're starting with, or if you want to challenge any part of this scope first (you should feel free to — I'd rather you disagree now than build something you don't actually understand the reasoning for).